Universidad Nacional
Vicerrectoría de Docencia
Programa Éxito Académico

Práctica de selección única

Marque con X la opción correcta

EIF201 Programación I · II Ciclo 2026 · Repaso para el Examen 2
Andrey Jesús Solano Rojas, tutor

Instrucciones

- Marque con una X la **única** opción correcta de cada pregunta, en la pregunta o en la hoja de respuestas de la última página.
- Resuelva sin computadora. Para las preguntas de trazado, dibuje la memoria o la tabla de valores antes de elegir.
- Salvo que se indique lo contrario, los fragmentos incluyen `#include <iostream>`, `#include <string>` y `using namespace std;`.
- Valor: un punto por pregunta (28 en total). Tiempo sugerido: 45 minutos.

Matrices automáticas

1. Con la matriz `m` mostrada, ¿qué imprime `cout << m[2][1] + m[1][3];`?

```
int m[3][4] = {{1, 2, 3, 4},
               {5, 6, 7, 8},
               {9, 10, 11, 12}};
```

- ☐ a) 11
- ☐ b) 18
- ☐ c) 15
- ☐ d) 16

2. Con la misma matriz `m`, ¿qué imprime este ciclo?

```
for (int c = 0; c < 3; c++)
    cout << m[c][c] << " ";
```

- ☐ a) 1 5 9
- ☐ b) 1 2 3
- ☐ c) 4 7 10
- ☐ d) 1 6 11

3. ¿Cómo se almacena en memoria una matriz automática `int m[3][4]`?

- ☐ a) Cada fila en un bloque de memoria independiente.
- ☐ b) En un bloque contiguo, columna por columna.
- ☐ c) En un bloque contiguo, fila por fila.
- ☐ d) Depende del compilador, no está definido.

4. ¿Cuál encabezado permite recibir la matriz `int m[3][4]` como parámetro?

- ☐ a) `void mostrar(int m[][], int filas)`
- ☐ b) `void mostrar(int** m, int filas)`
- ☐ c) `void mostrar(int m[3][], int filas)`
- ☐ d) `void mostrar(int m[][4], int filas)`

Matrices dinámicas

5. ¿Qué imprime el siguiente fragmento?

```
int** d = new int*[2];
for (int f = 0; f < 2; f++) {
    d[f] = new int[3];
    for (int c = 0; c < 3; c++)
        d[f][c] = f * 3 + c;
}
cout << d[0][1] + d[1][1];
```

- ☐ a) 5
- ☐ b) 3
- ☐ c) 4
- ☐ d) 7

6. ¿Cuál es la forma correcta de liberar la matriz `int** d` de `F` filas?

- ☐ a) `delete[] d;` y después `for (f) delete[] d[f];`
- ☐ b) `delete d;`
- ☐ c) `for (f) delete[] d[f];` y después `delete[] d;`
- ☐ d) `for (f) delete d[f];` y después `delete d;`

7. En una clase que declara `Reserva*** asientos;`, ¿de qué tipo es `asientos[1]`?

- ☐ a) `Reserva**`
- ☐ b) `Reserva*`
- ☐ c) `Reserva***`
- ☐ d) `Reserva`

8. El constructor crea la matriz de punteros a objetos pero no inicializa las celdas en `nullptr`. ¿Qué ocurre cuando el destructor hace `delete` de cada celda?

- ☐ a) Comportamiento indefinido: se intentan liberar direcciones basura.
- ☐ b) Error de compilación.
- ☐ c) Nada, `new` deja las celdas en `nullptr` automáticamente.
- ☐ d) Se liberan solo las celdas ocupadas.

Herencia

9. ¿Qué imprime el siguiente fragmento?

```
class A {
public:
    A() { cout << "A "; }
    ~A() { cout << "~A "; }
};
class B : public A {
public:
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
};
// En main:
{ B b; }
```

- ☐ a) B A A B
- ☐ b) A B B A
- ☐ c) B A B A
- ☐ d) A B A B

10. Con herencia pública, ¿a qué miembros de la clase base puede acceder directamente la clase derivada?

- ☐ a) A todos, incluidos los `private`.
- ☐ b) Solo a los `protected`.
- ☐ c) A los `public` y `protected`.
- ☐ d) Solo a los `public`.

11. ¿Qué sucede al compilar este código?

```
class Padre {
    int x;
public:
    Padre(int x) : x(x) {}
};
class Hija : public Padre {
    int y;
public:
    Hija(int y) : y(y) {}
};
```

- ☐ a) Compila; `x` queda con basura.
- ☐ b) Compila con una advertencia.
- ☐ c) Compila; `x` queda en 0.
- ☐ d) No compila: `Padre` no tiene constructor sin parámetros.

12. ¿Qué imprime `cout << h.f();`?

```
class Padre {
public:
    virtual int f() const { return 10; }
};
class Hija : public Padre {
public:
    int f() const override { return Padre::f() + 5; }
};
// En main:
Hija h;
```

- ☐ a) 15
- ☐ b) Recursión infinita.
- ☐ c) 5
- ☐ d) 10

Polimorfismo

13. ¿Qué imprime el siguiente fragmento?

```
class Animal {
public:
    void presentarse() const { cout << "animal "; }
    virtual void sonido() const { cout << "... "; }
    virtual ~Animal() {}
};
class Perro : public Animal {
public:
    void presentarse() const { cout << "perro "; }
    void sonido() const override { cout << "guau "; }
};
// En main:
Perro p;
Animal* a = &p;
a->presentarse();
a->sonido();
```

- ☐ a) animal ...
- ☐ b) perro guau
- ☐ c) animal guau
- ☐ d) perro ...

14. Con las clases de la pregunta anterior, ¿qué imprime `Animal copia = p; copia.sonido();`?

- ☐ a) Comportamiento indefinido.
- ☐ b) `...`
- ☐ c) No compila.
- ☐ d) `guau`

15. `Figura` tiene el método `virtual double area() const = 0;` y `Circulo` lo implementa. ¿Cuál línea **no** compila?

- ☐ a) `Circulo* c = new Circulo(2);`
- ☐ b) `Figura* f = new Circulo(2);`
- ☐ c) `Figura f;`
- ☐ d) `Circulo c(2); Figura& r = c;`

16. ¿Qué ocurre con este código?

```
class Figura {
public:
    virtual double area() const = 0;
    virtual ~Figura() {}
};
class Circulo : public Figura {
    double r;
public:
    Circulo(double r) : r(r) {}
    double area() { return 3.14 * r * r; }
};
// En main:
Figura* f = new Circulo(2);
```

- ☐ a) Compila y `f->area()` retorna 12.56.
- ☐ b) No compila: `Circulo` sigue siendo abstracta.
- ☐ c) No compila: falta la palabra `virtual` en `Circulo`.
- ☐ d) Compila, pero `f->area()` retorna 0.

17. El destructor de `Base` no es virtual. ¿Qué se observa normalmente al ejecutar `Base* p = new Derivada(); delete p;`, si cada destructor imprime su nombre?

- ☐ a) `~Base`
- ☐ b) `~Base ~Derivada`
- ☐ c) No compila.
- ☐ d) `~Derivada ~Base`

18. ¿Cuál afirmación es correcta?

- ☐ a) La sobrecarga se decide en ejecución según el tipo del objeto.
- ☐ b) La sobrecarga solo es posible entre una clase base y una derivada.
- ☐ c) Un método `static` puede ser `virtual`.
- ☐ d) La sobrescritura requiere un método `virtual` con la misma firma en la base.

19. ¿Qué imprime `cout << e->toString();`?

```
class Empleado {
public:
    virtual string getTipo() const { return "Empleado"; }
    string toString() const { return "[" + getTipo() + "]"; }
    virtual ~Empleado() {}
};
class PorHoras : public Empleado {
public:
    string getTipo() const override { return "Por horas"; }
};
// En main:
Empleado* e = new PorHoras();
```

- ☐ a) `[]`
- ☐ b) `[Por horas]`
- ☐ c) No compila: `toString` no es virtual.
- ☐ d) `[Empleado]`

20. ¿Qué imprime este fragmento?

```
Figura* v[3] = { new Rectangulo(2, 3),
                 new Cuadrado(3),      // hereda de Rectangulo
                 new Triangulo(4, 5) }; // area = b * h / 2
double total = 0;
int n = 0;
for (int i = 0; i < 3; i++) {
    total += v[i]->area();
    if (dynamic_cast<Rectangulo*>(v[i]) != nullptr)
        n++;
}
cout << total << " " << n;
```

- ☐ a) 19 1
- ☐ b) 25 1
- ☐ c) 35 2
- ☐ d) 25 2

Casting

21. Si `Reserva* r = new ReservaGeneral(...);`, ¿qué retorna `dynamic_cast<ReservaVIP*>(r)`?

- ☐ a) No compila.
- ☐ b) `nullptr`.
- ☐ c) Lanza la excepción `bad_cast`.
- ☐ d) Un puntero válido a una `ReservaVIP`.

22. Con la misma `r`, ¿qué hace `dynamic_cast<ReservaVIP&>(*r)`?

- ☐ a) Lanza la excepción `std::bad_cast`.
- ☐ b) Retorna una referencia a la reserva general.
- ☐ c) No compila con referencias.
- ☐ d) Retorna una referencia nula.

23. ¿Cuál es la diferencia principal entre `dynamic_cast` y `static_cast` al convertir de base a derivada?

- ☐ a) `dynamic_cast` solo sirve para convertir de derivada a base.
- ☐ b) `static_cast` verifica el tipo en ejecución y `dynamic_cast` no.
- ☐ c) Son equivalentes; solo cambia la sintaxis.
- ☐ d) `dynamic_cast` verifica el tipo real en ejecución; `static_cast` no verifica nada.

24. ¿Qué ocurre con este código?

```
class Base {
public:
    ~Base() {}
};
class Derivada : public Base {};
// En main:
Base* b = new Derivada();
Derivada* d = dynamic_cast<Derivada*>(b);
```

- ☐ a) Lanza `bad_cast`.
- ☐ b) Compila y `d` es `nullptr`.
- ☐ c) No compila: `Base` no es una clase polimórfica.
- ☐ d) Compila y `d` es un puntero válido.

25. `Gerente` hereda de `EmpleadoFijo`. ¿Qué retorna la conversión de la segunda línea?

```
EmpleadoFijo* e = new Gerente("3-3333-3333", "Marta", 900000, 250000);
Gerente* g = dynamic_cast<Gerente*>(e);
```

- ☐ a) Lanza `bad_cast`.
- ☐ b) Un puntero válido.
- ☐ c) No compila.
- ☐ d) `nullptr`, porque `e` es de tipo `EmpleadoFijo*`.

26. ¿Cuál conversión es un *upcasting* y por qué no necesita `dynamic_cast`?

- ☐ a) `Base* b = d;` con `Derivada* d`: es implícita porque toda derivada es una base.
- ☐ b) `Derivada* d = b;` con `Base* b`: es implícita porque comparten atributos.
- ☐ c) Ninguna; toda conversión entre clases requiere un cast explícito.
- ☐ d) `Derivada* d = b;`: solo compila con `static_cast`.

Listas enlazadas

27. La lista es `4 -> 9 -> 2 -> 7 -> 5`. ¿Qué imprime este fragmento?

```
int n = 0;
Nodo* a = primero;
while (a->sig != nullptr) {
    if (a->dato > a->sig->dato)
        n++;
    a = a->sig;
}
cout << n << " " << a->dato;
```

- ☐ a) 2 7
- ☐ b) 3 5
- ☐ c) Se cae al llegar al final.
- ☐ d) 2 5

28. Un método elimina el primer nodo con `delete primero;` sin actualizar antes `primero`. ¿Qué consecuencia tiene?

- ☐ a) Se libera toda la lista.
- ☐ b) Ninguna; el siguiente nodo pasa a ser el primero automáticamente.
- ☐ c) `primero` apunta a memoria liberada y se pierde el resto de la lista.
- ☐ d) Error de compilación.

Hoja de respuestas

Nombre: _____ Puntaje: _____ / 28

N.º	a	b	c	d	**N.º**	a	b	c	d
1	☐	☐	☐	☐	15	☐	☐	☐	☐
2	☐	☐	☐	☐	16	☐	☐	☐	☐
3	☐	☐	☐	☐	17	☐	☐	☐	☐
4	☐	☐	☐	☐	18	☐	☐	☐	☐
5	☐	☐	☐	☐	19	☐	☐	☐	☐
6	☐	☐	☐	☐	20	☐	☐	☐	☐
7	☐	☐	☐	☐	21	☐	☐	☐	☐
8	☐	☐	☐	☐	22	☐	☐	☐	☐
9	☐	☐	☐	☐	23	☐	☐	☐	☐
10	☐	☐	☐	☐	24	☐	☐	☐	☐
11	☐	☐	☐	☐	25	☐	☐	☐	☐
12	☐	☐	☐	☐	26	☐	☐	☐	☐
13	☐	☐	☐	☐	27	☐	☐	☐	☐
14	☐	☐	☐	☐	28	☐	☐	☐	☐